Drools
explained simply

What a rule engine is and how it decides.



Meriem Trabelsi

Software engineer

- **The idea**
- How it decides
- In practice

Part 1

The idea

What problem does a rule engine solve?



Business rules often hide in code

```
if (customer.years >= 3
    && order.discount == 0) {
  order.setDiscount(10);
} else if (...) {
  // and many more
}
```

A rule changes

a developer edits the code

It waits

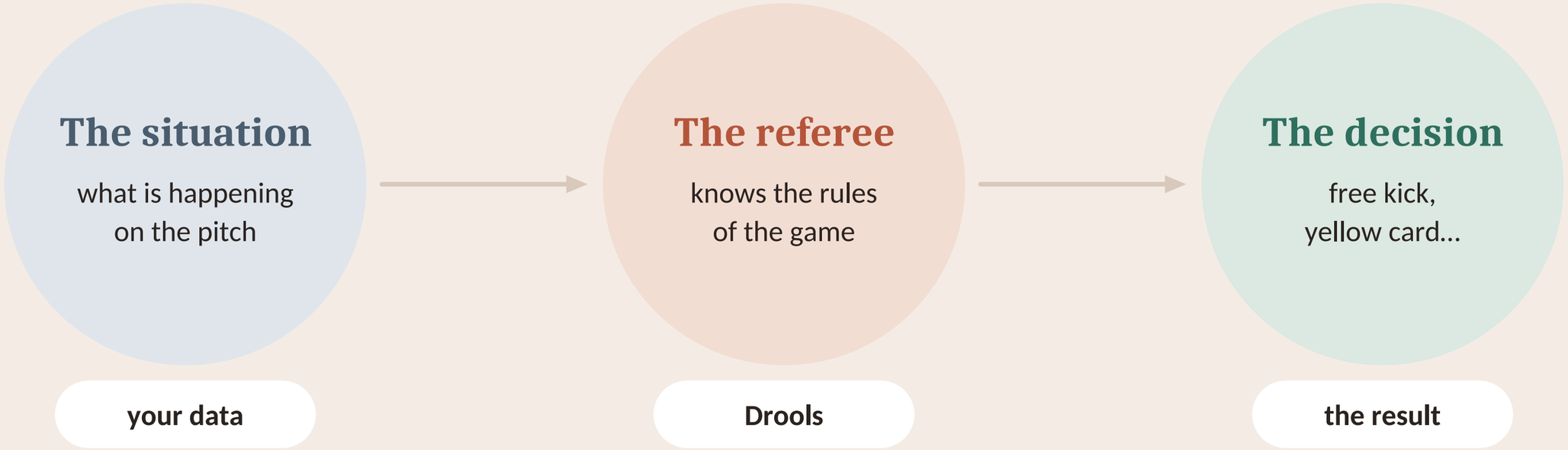
for the next release

Hard to read

for anyone outside the team

The rule is business policy. The code is the wrong place to keep it.

Think of a referee



You describe the situation. The engine applies the rules.

Three words to learn

1

Fact

A piece of data you give the engine.

a Customer, an Order

2

Rule

When something is true, do something.

loyal customer: discount

3

Session

Where facts and rules meet.

insert, then fire

Read a rule like a sentence

```
rule "Loyalty discount"
when
  $c : Customer( years >= 3 )
  $o : Order( customer == $c,
              discount == 0 )
then
  modify( $o ) { setDiscount( 10 ) }
end
```

In plain words

a customer who has been loyal for 3 years or more

… and an order of theirs with no discount yet

give that order 10 % off

when = the conditions. then = the action.

What your Java code does

1

Give the facts

```
session.insert(customer);
```

add data to the session

2

Let it decide

```
session.fireAllRules();
```

the engine picks and runs rules

3

Read the result

```
order.getDiscount();
```

your objects were updated

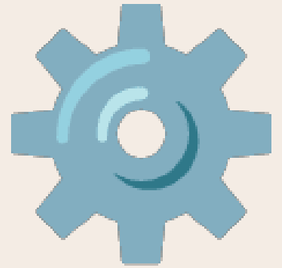
Your code never names a rule. The engine chooses.

The idea
How it decides
In practice

Part 2

How it decides

How does it find the right rule so fast?



The naive way: test everything

the problem

1 rule

with 2 conditions

1,000 × 1,000

customers × orders

1 million

checks, after every change

Add a third condition: 1 billion. Real systems have hundreds of rules.

Charles Forgy's answer (1982): the Rete algorithm.

The Rete algorithm

Forgy, 1982

1

Compile

The rules become a network, built once. A condition shared by many rules is one node.

2

Test

Each new fact checks its own conditions, once, whatever the number of rules.

3

Remember

Facts and partial matches that pass are kept in memory.

4

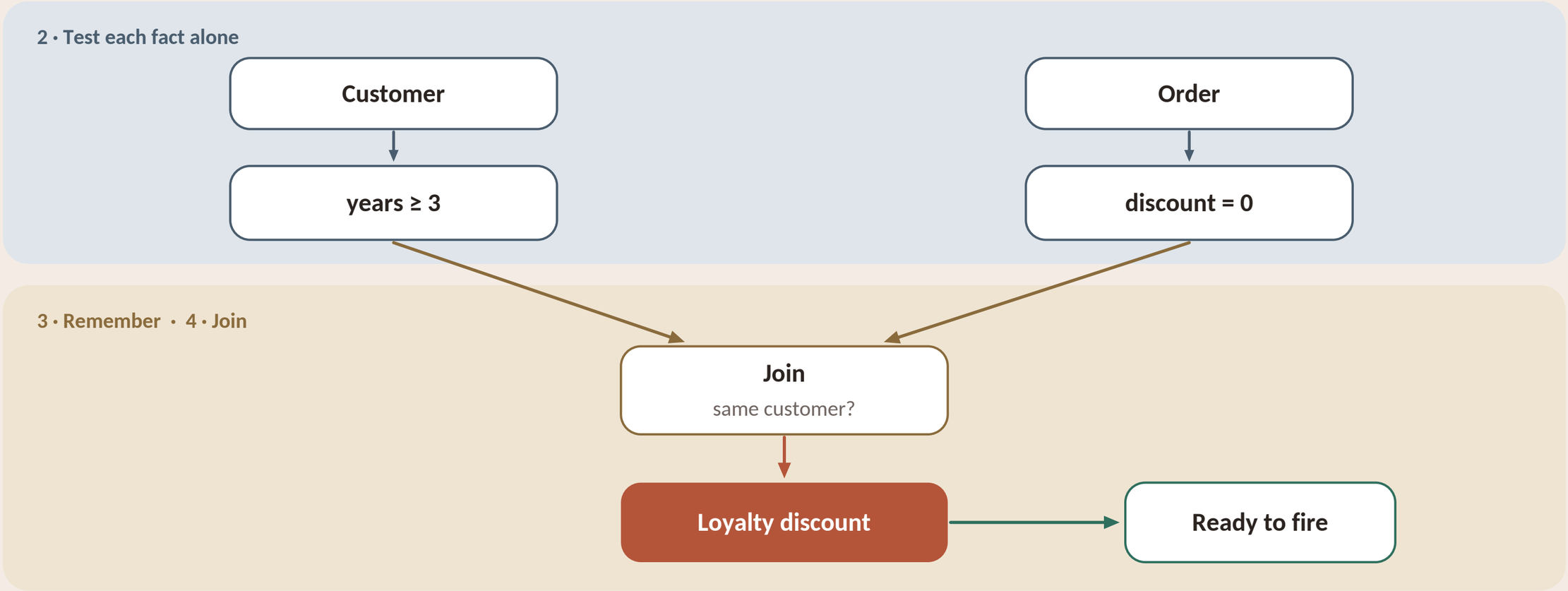
Join

Facts are combined. A complete match makes the rule ready to fire.

Only what changed is processed. Everything else is remembered.

Rete: the network for one rule

the network



Rules are compiled once into a network. Facts flow through it.

Follow one new customer

step by step

1

Arrives

inserted in
the session

2

Tested

years ≥ 3?
yes

3

Joined

with their order,
already in memory

4

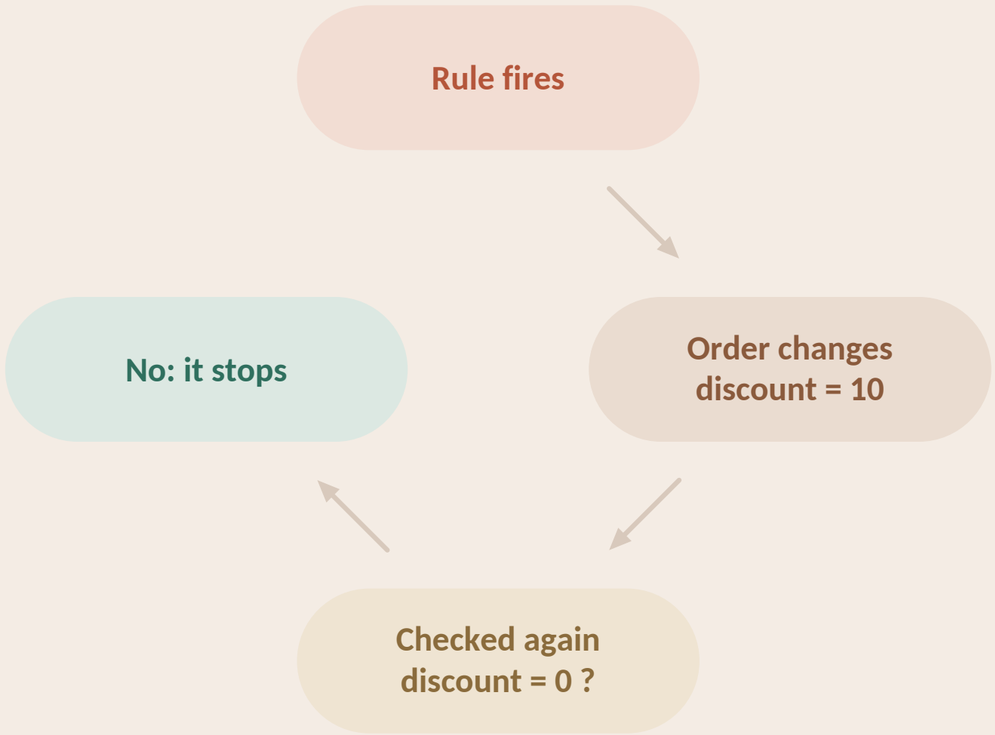
Ready

the rule waits
to fire

Nothing else moves. Everyone else is already known.

Why the rule fires only once

no loop



The guard

"discount == 0" is false after the change.

So the rule cannot match again.

Each action should make its own condition false.

Phreak: wait before working

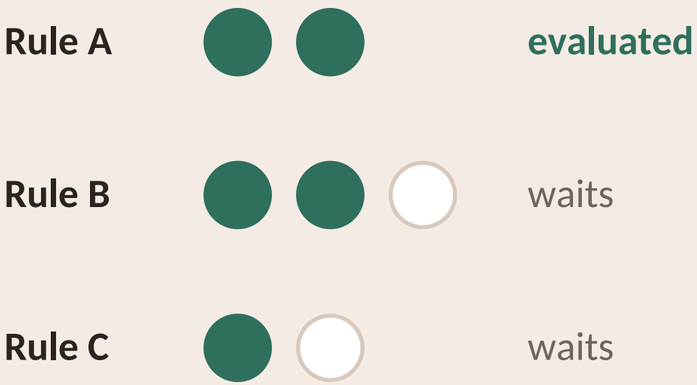
today

Like a careful cook

Starts a recipe only when every ingredient is on the table.

No ingredient, no work.

One light per condition



Drools uses Phreak by default since version 6.

Rete works eagerly. Phreak waits until a rule can really fire.

The agenda: who goes first

order

1. **Refuse an invalid order** — `salience 100`
2. **Loyalty discount** — `salience 50`
3. **Send a thank-you email** — `salience 10`

Salience

A number on the rule.

Higher fires first.

Use it rarely: good rules do not depend on order.

When several rules are ready, the agenda orders them.

- The idea
- How it decides
- **In practice**

Part 3

In practice

When is a rule engine worth it?



Who uses Drools?



Open source, under the Apache KIE project.

Free to use in any Java application.

Built on Drools

- **Red Hat Decision Manager** — *now IBM BAMOE*
- **jBPM** — *business process engine*
- **Kogito** — *cloud-native rules and processes*

Rule engines are common in

decisions that follow written policy

Banking
credit and fraud checks

Insurance
claims and pricing

Telecom
offers and eligibility

Behind many business decisions, there is a rule engine.

Use it or not?

Good fit

+ Many rules
+ Rules change often
+ Business people read them
+ Rules interact with each other

Plain code is simpler

– A handful of rules
– They almost never change
– No time to write tests
– Nobody knows the engine

A rule engine pays off when rules are many and change often.

Four good habits

1

Test every rule

insert facts, fire, check

2

Connect every condition

or you multiply the data

3

Guard every action

or the rule repeats

4

Keep sessions short

one decision, then close

Recap and glossary

1. A rule engine keeps decisions out of the code.
2. You give facts; it finds the rules that match.
3. Rete: test once, remember matches.
4. Phreak: only work when a rule can fire.
5. The agenda decides the order.

Glossary

Fact	data given to the engine
Working memory	the facts it holds
Activation	a rule ready to fire
Agenda	the queue of activations
Rete / Phreak	the matching algorithms

Next week

DMN explained simply

A business decision written as a table anyone can read.

Follow my posts on

mariem-trabelsi.github.io